iOS SDK Introduction

Introduction **2**
Setup the SPM dependency 2
Implementation **3**
Getting started 3
Customization 4
Example code 4
Authentication 5
Data flow 5
Example code 6
Open a book 7
Example code 7
Data flow 8
Additional info **9**

Introduction

This document is a brief introduction to the WeDoBooks iOS SDK, including setup and usage. Presented as a step-by-step guide in Xcode along with a couple of diagrams illustrating data flow.

Please note that this document describes version 1.1.0 of the iOS SDK

Setup the SPM dependency

To use the SDK in a host application, open the Xcode project of the application that will be using the SDK and navigate to the Package Dependencies tab of the project settings.

Click the “+” button and enter the URL of the GitHub repository hosting the SPM package for the SDK in the search bar: https://github.com/wedobooks/wedobooks-sdk-ios-spm

When Xcode has located the package, select the dependency rule and the semantic version you prefer. We recommend using “Exact Version” with the newest version available.

Click “Add Package” and let Xcode install all the necessary dependencies.

Implementation

Getting started

This section describes the basics of how to implement usage of the SDK in your application. The first thing that needs to happen is setting up the SDK.

To do that, first import the necessary Swift modules: `FirebaseProvider` and `WeDoBooksSDK`.

```
None
import FirebaseProvider
import WeDoBooksSDK
```

The `setup` method requires 4 arguments which are the credentials for the ebook reader component used in the SDK, the Mode of the SDK and a `FirebaseAdapterFactory` instance. Mode can be `.streaming` or `.library`. WeDoBooks will inform you of which one to use. The call to `setup` looks like this.

```
None
try! WeDoBooksFacade.shared.setup(
    readerKey: readerKey,
    readerSecret: readerSecret,
    mode: .streaming,
    firebaseAdapterFactory: FirebaseAdapterFactory()
)
```

If you're using Firebase in the host app, then you provide a path to the SDK Firebase config file in the setup call.

```
None
let sdkGoogleInfoFileName = "GoogleService-Info-SDK.plist"
guard let sdkGoogleInfoFilePath = Bundle.main.path(
   forResource: sdkGoogleInfoFileName,
   ofType: nil
) else {
   fatalError("Path in bundle for SDK Google Info plist file not found")
}

try! WeDoBooksFacade.shared.setup(
    readerKey: readerKey,
    readerSecret: readerSecret,
```

```
    mode: .library,
    firebaseAdapterFactory: FirebaseAdapterFactory(),
    firebaseConfigFilePath: sdkGoogleInfoFilePath
)
```

We recommend doing this as soon as possible at runtime. For example in the app delegate or top level view controller. Calling `setup` more than once will result in an error.

Customization

The SDK exposes APIs for customization of certain parts of the user interface such as certain colors, texts, icons and event handling.

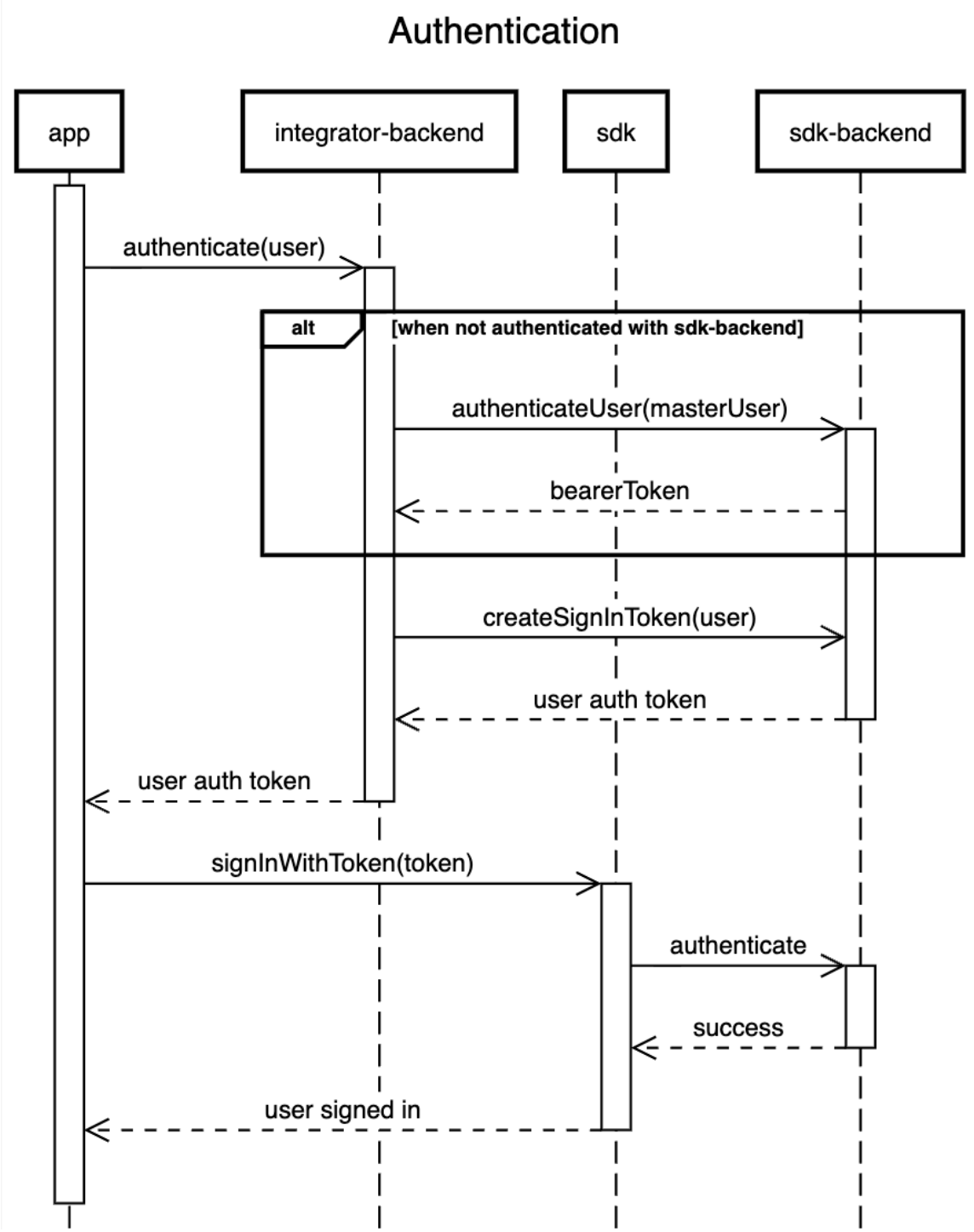
Example code

```
None
WeDoBooksFacade.shared.localization.setLanguage(.english)
let localizations: [WeDoBooksFacade.Localization.LocalizationKeys :
[WeDoBooksFacade.Localization.Language : String]] = [
    .buttonSave : [.english : "Custom save"],
    .playerPlaybackRateReset : [.english : "Custom reset"],
    .playerPlaybackRateSpeed : [.english : "Custom speed"],
    .playerMoreMenuAboutBookLabel : [.english : "Custom about book"],
]

WeDoBooksFacade.shared.localization.setCustomLocalizations(localizations)

WeDoBooksFacade.shared.configuration.showFinishEbookButton = false
WeDoBooksFacade.shared.configuration.showFinishAudiobookButton = false
WeDoBooksFacade.shared.configuration.showAboutAudioBookButton = false

WeDoBooksFacade.shared.configuration.allowEbookDownloadUsingMobileData =
true

WeDoBooksFacade.shared.images.icons.set(.close, to: "sf:xmark.app")
WeDoBooksFacade.shared.images.icons.set(.down, to: "down-alt")
```

Authentication

Most SDK features require an authenticated user and will throw an error if there's no user authenticated. Please note that user authentication requires a backend-to-backend integration.

Data flow



Example code

```
None
Task {
    let token = "<some-token-obtained-through-your-own-backend>"
    let result = await WeDoBooksFacade.shared.userOperations.signIn(with:
token)
    switch result {
    case .success(let user):
        print("Sign in success: \(user)")
    case .failure(let error):
        print("Failure: \(error)")
    }
}
```

Open a book

The WeDoBooksSDK uses a concept called `Checkout` when opening books. A checkout is basically just an association between a user and a specific book.

To open a book you must first get hold of a `Checkout` value. To do that you call `WeDoBooksSDK.shared.bookOperations.checkoutBook(with: “9788702073416”)`

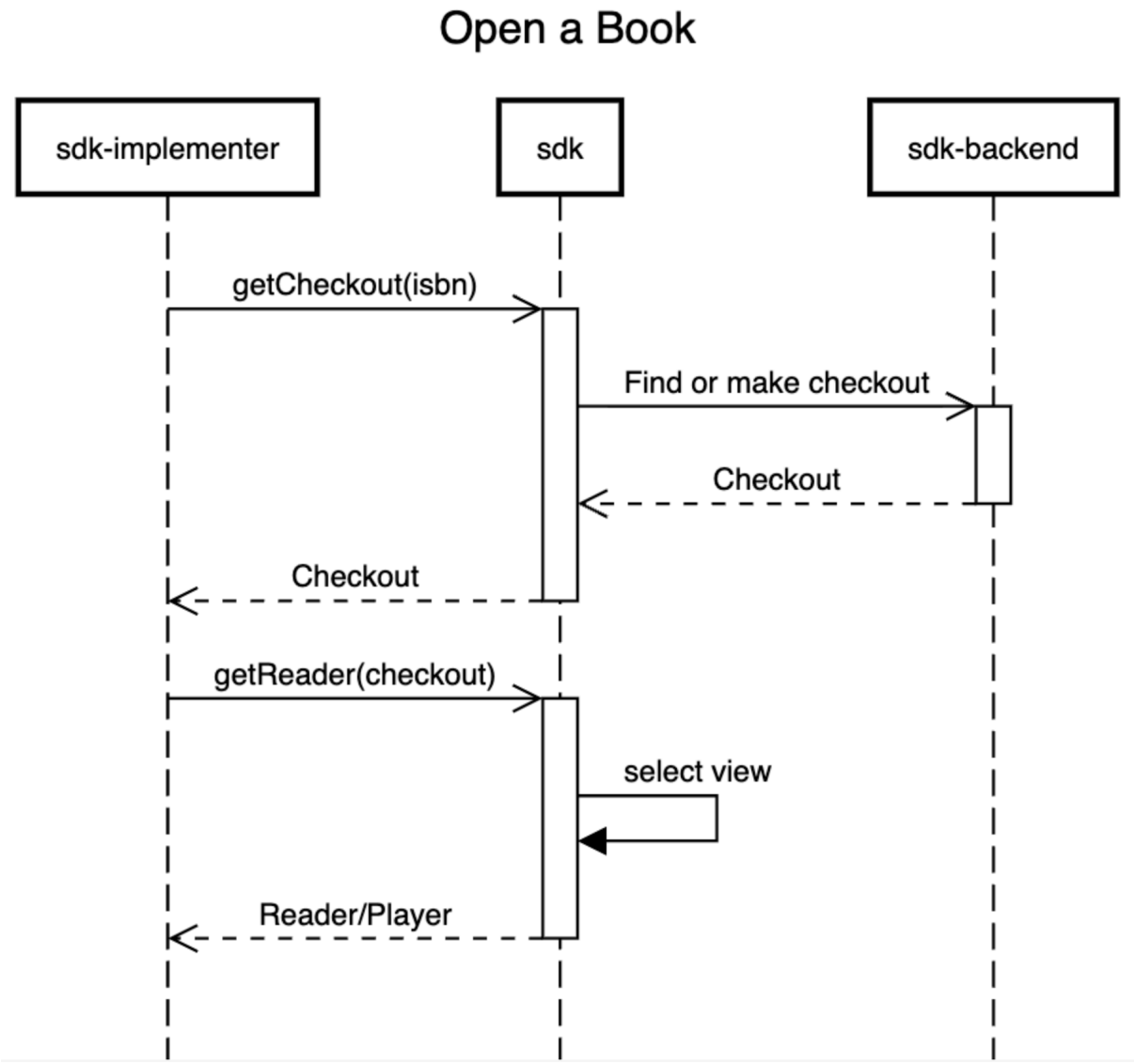
The call either creates a new checkout or returns an existing one if the user already has a checkout for the given ISBN.

After obtaining the checkout, the reader can be opened. The SDK automatically launches either the audio player or the e-book reader based on the ISBN. The reader or player screens are shown as full screen modals so you have to pass a presenting view controller instance to the `openCheckout(...)` call.

Example code

```
None
Task {
    let checkoutResult = await
WeDoBooksFacade.shared.bookOperations.checkoutBook(with: "9788702073416")
    switch checkoutResult {
    case .success(let checkout):
        do {
            // Call like this if you want to use WeDoBooks cover
            try WeDoBooksFacade.shared.bookOperations.openCheckout(checkout,
presentedBy: self)

            // Call like this if you want to use your own cover image
            let coverImage: UIImage = ....
            try WeDoBooksFacade.shared.bookOperations.openCheckout(checkout,
presentedBy: self), customCover: .image(coverImage))

            // Call like this if you want to use your own cover url
            let url: URL = ....
            try WeDoBooksFacade.shared.bookOperations.openCheckout(checkout,
presentedBy: self), customCover: .url(url))
        } catch {
            print("Open checkout failed: \(error)")
        }
    case .failure(let error):
        print("Checkout book failed: \(error)")
    }
}
```

Data flow


Open a Book
sdk-implementer
sdk
sdk-backend
getCheckout(isbn)
Find or make checkout
Checkout
Checkout
getReader(checkout)
select view
Reader/Player

Headless Audio Player

New in version 1.1.0 is the capability of playing an audiobook without showing the built in Player UI and instead use your own UI.

This is done through the `headlessAudiobook` namespace on the `WeDoBooksFacade` singleton object. You first need to load a checkout and then you can perform play, pause, seek and other operations on the loaded book.

Example code

```
None
Task { @MainActor in
    guard let checkout = await ensureDemoAudiobookCheckout() else { return }

    do {
        try WeDoBooksFacade.shared
            .headlessAudioPlayer
            .loadBook(book: checkout, startPosition: startPosition)

        try WeDoBooksFacade.shared
            .headlessAudioPlayer
            .play()
    } catch {
        print("Headless audio player error: \(error)")
    }
}
```

Additional info

The SDK provides more than obtaining book checkouts and launching the reader. For concrete examples, refer to the sample application, which demonstrates obtaining user reading statistics and the implementation of a “currently playing” bar.

For additional customization options and integration points, consult the backend SDK API documentation. The API exposes extra endpoints and configuration for checkouts and reading statistics.